Enterprise IAM Integration Lab — SCIM 2.0 Provisioning, Authentication & Troubleshooting

Farhan Muntasir · **Python 3.13** · **Flask** · **SQLite** · **PowerShell** · **SCIM 2.0**
Lab date: November 1, 2026 · **Result:** Local API and PATCH persistence verified

> **What this repository teaches:** How a provisioning client sends identity changes to a target application; how bearer-token security, Flask request handling, SCIM JSON, and SQLite fit together; and how to debug failures at each layer.
>
> **Scope:** A locally tested SCIM-style server, **not** an active Microsoft Entra ID provisioning integration or production-ready SCIM implementation.

Table of contents

1. Why this lab matters
2. Architecture: understand the components
3. How an API request is processed
4. Prepare and start the project
5. Authentication: why 401 happened
6. Create a user with POST
7. Read users with GET
8. Update Jane with PATCH
9. Verify changes in SQLite
10. Complete troubleshooting guide
11. IAM lifecycle and results
12. Security, limitations and next steps

1. Why this lab matters

When a person joins an organization, changes their surname, or leaves, downstream applications must reflect those changes. **SCIM (System for Cross-domain Identity Management)** defines a common HTTP/JSON model for user and group identity management. A provisioning client can call a target application's SCIM API instead of an administrator updating every account manually.

Real-world example: Jane Smith changes her surname to Johnson. A client sends a SCIM `PATCH` request to the application's user endpoint. The target checks authentication, validates the requested attribute, updates the database, and returns the modified identity. In this lab, **PowerShell/curl simulated the provisioning client**.



What each stage means: Joiner = create a new account (`POST`); **Mover** = modify an existing account (`PATCH`); **Leaver** = disable or remove an account (`PATCH active=false` or `DELETE`). In this project, POST and surname PATCH were verified; a complete leaver workflow was not verified.

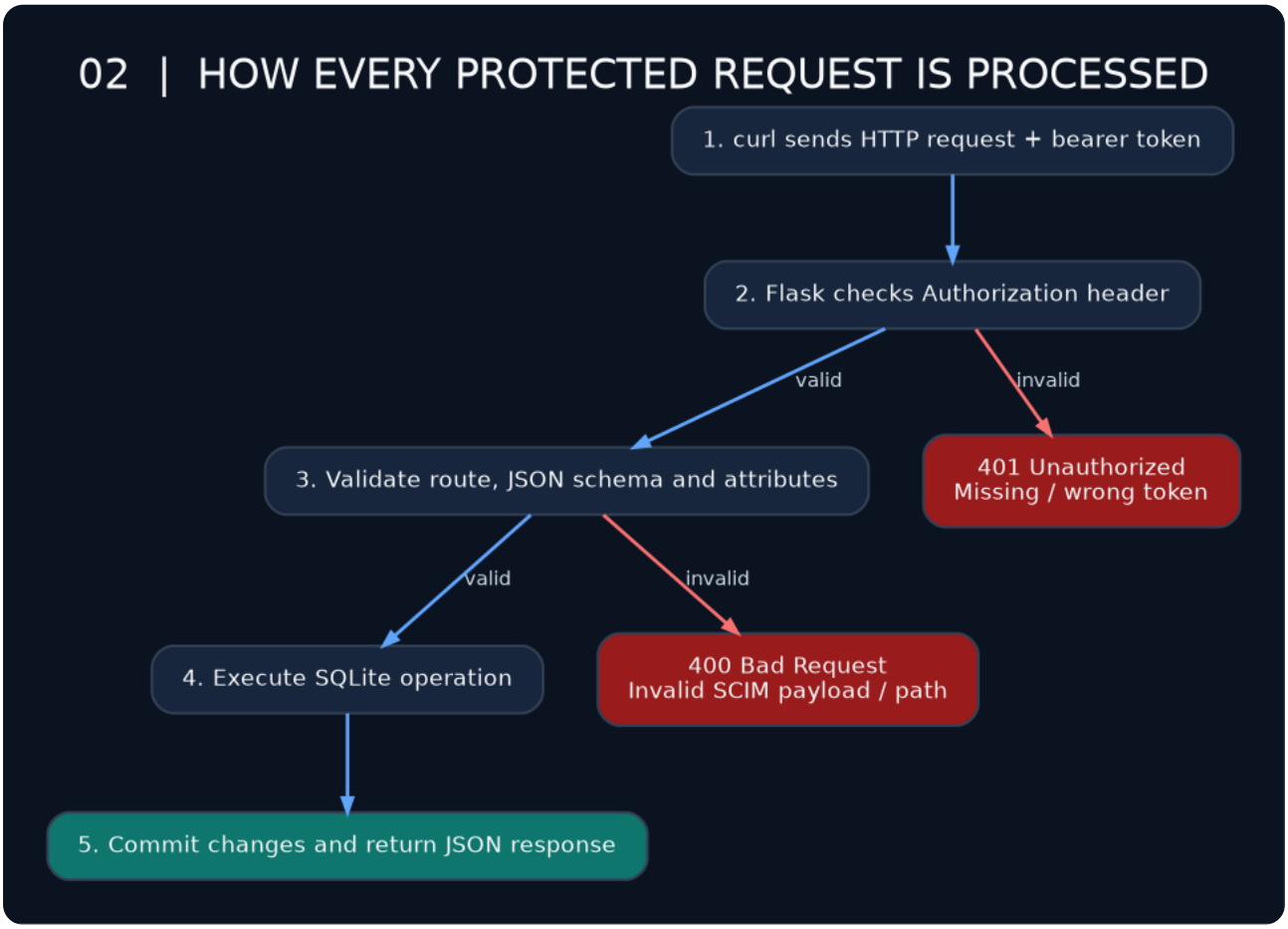
2. Architecture: understand the components



Component	What it does	Why it matters
PowerShell / curl.exe	Sends HTTP requests and JSON	Simulates a provisioning client such as an identity platform
Flask (`app.py`)	Routes requests, checks tokens and validates payloads	Acts as the target application's SCIM API
SQLite (`scim_users.db`)	Stores account IDs, names, emails and active state	Makes user changes persist beyond one HTTP response
Bearer token	Shared secret in the `Authorization` header	Rejects unauthorized API requests
SCIM JSON	Standardized schema and PATCH operation structure	Makes identity attributes understandable across systems

Important: The diagram represents your **actual local test**. Microsoft Entra ID is a *future integration target*, not a component connected during this lab. The local API ran on `http://127.0.0.1:5004` (HTTP is acceptable for an isolated localhost exercise; a real integration needs HTTPS).

3. How an API request is processed



1. **Client sends a request:** `curl.exe` specifies the HTTP method, URL, authorization header, and optional JSON body.
2. **Authentication:** Flask checks the bearer token. Missing or mismatched credentials return **401 Unauthorized**.
3. **Validation:** Flask checks the JSON structure and supported attributes. Unsupported PATCH paths can return **400 Bad Request**.
4. **Database operation:** A GET selects rows; POST inserts a user; PATCH updates selected columns.
5. **Response:** Flask returns SCIM JSON and an HTTP status code. A separate SQLite query confirms that updates were committed.

Endpoint reference

Method	Route	Purpose	Lab evidence
`GET`	`/`	Local status page	Seen running
`GET`	`/scim/v2/Users`	List users	**200 verified**
`POST`	`/scim/v2/Users`	Create a user	**201 verified**
`GET`	`/scim/v2/Users/{id}`	Get one user	**200 verified**
`PATCH`	`/scim/v2/Users/{id}`	Update supported attributes	**200 verified**
`DELETE`	`/scim/v2/Users/{id}`	Delete a user	Route registered; end-to-end result not verified
`GET`	`/scim/v2/ServiceProviderConfig`	Advertise server capabilities	Route registered

These are **implemented routes**, not proof of complete RFC 7643/7644 compatibility.

4. Prepare and start the project

Files

```
entra-scim-lab/
├── app.py                    # Flask SCIM routes and auth
├── employee.json             # New-user POST payload
├── update_employee.json      # Surname PATCH payload
├── scim_users.db             # Local SQLite data (do not
publish)
```

```
├── venv/                    # Python environment (do not
publish)
└── README.md
```

PowerShell window A: server

```
cd C:\Users\FOXTEC\entra-scim-lab
# Only if the virtual environment does not exist:
# py -m venv venv
.\venv\Scripts\python.exe -m pip install flask
.\venv\Scripts\python.exe -m py_compile app.py
$env:SCIM_BEARER_TOKEN = 'replace-with-your-own-local-test-
secret'
.\venv\Scripts\python.exe -m flask --app app run --host
127.0.0.1 --port 5004 --no-reload --no-debugger
```

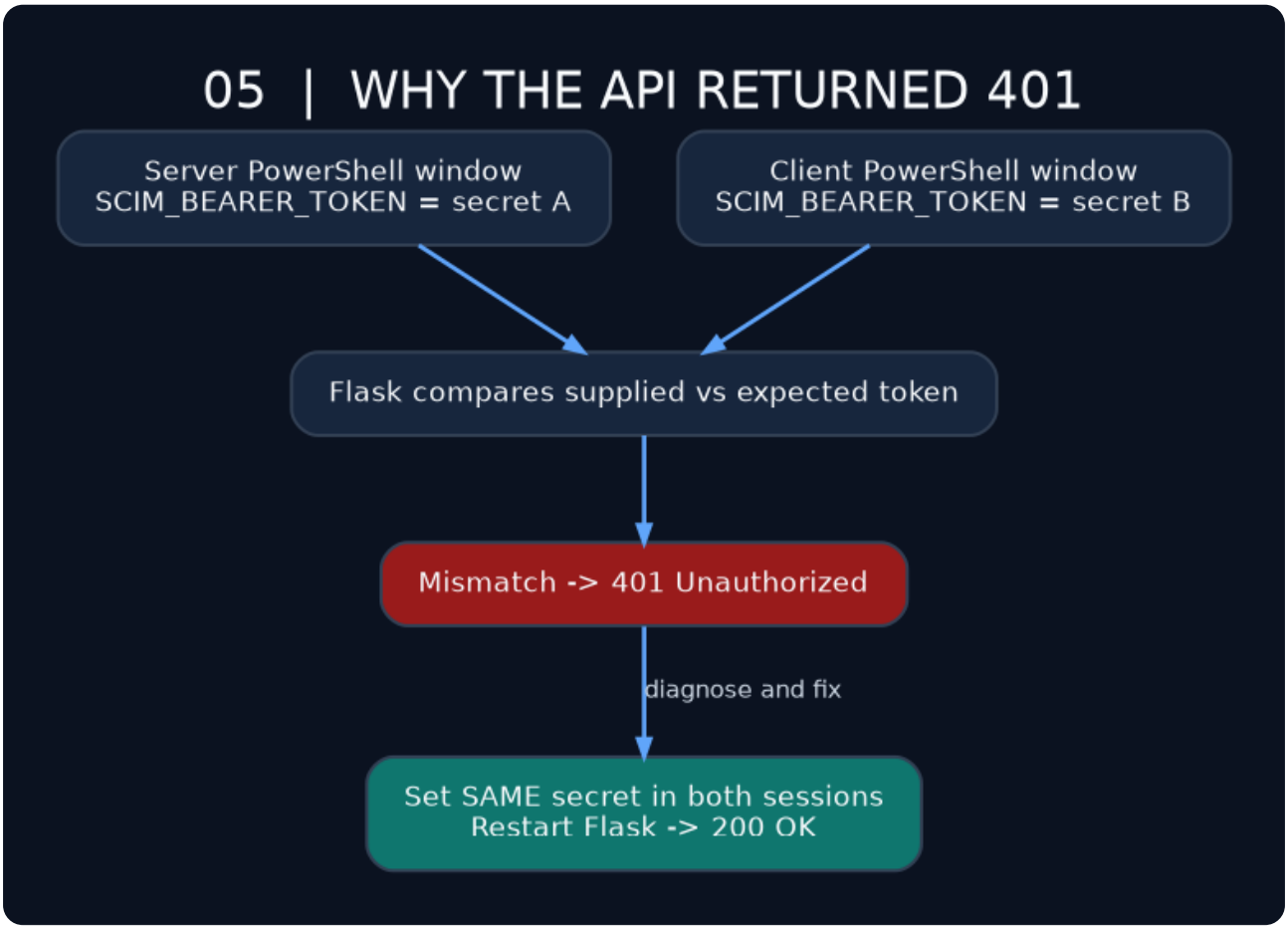
Expected: `Running on http://127.0.0.1:5004`. Keep this window open. The `py_compile` command detects syntax and indentation problems *before* you start Flask.

PowerShell window B: client

```
cd C:\Users\FOXTEC\entra-scim-lab
$env:SCIM_BEARER_TOKEN = 'replace-with-your-own-local-test-
secret'
$env:SCIM_BEARER_TOKEN.Length
curl.exe -i -H "Authorization: Bearer $env:SCIM_BEARER_TOKEN"
http://127.0.0.1:5004/scim/v2/Users
```

Expected: `HTTP/1.1 200 OK` and a SCIM `ListResponse`. Both windows must use the **same** secret. Do not print, share or commit a real token.

5. Authentication: why 401 happened



Your lab initially returned `HTTP/1.1 401 UNAUTHORIZED` repeatedly, even while the Flask server was running. The key distinction is **a running server is not the same as an authenticated request**. A PowerShell environment variable is local to that process; changing it in one window does not update another already-running Python process.

How we isolated the issue:

```
# Check token presence without revealing it
$env:SCIM_BEARER_TOKEN.Length

# Find the process listening on the API port
Get-NetTCPConnection -LocalPort 5004 -State Listen
-ErrorAction SilentlyContinue |
  Select-Object LocalAddress,LocalPort,OwningProcess
```

```
# Replace 27384 with the current owning process ID
Get-CimInstance Win32_Process -Filter "ProcessId = 27384" |
  Select-Object ProcessId,ParentProcessId,Name,CommandLine
```

Once we identified the correct Flask process, we restarted the server with the intended token configured in its own session, and configured the matching token in the client. **Result: GET `/Users` returned 200 OK.**

Negative security test: Sending `Authorization: Bearer invalid-test-token` returned **401 Unauthorized**, confirming that the API rejects invalid credentials.

6. Create a user with POST



Scenario: Jane Smith joins the company. The provisioning client sends her user attributes in a SCIM User resource.

`employee.json`:

```
{
  "schemas": ["urn:ietf:params:scim:schemas:core:2.0:User"],
  "userName": "jane.smith@example.com",
  "name": {"givenName": "Jane", "familyName": "Smith"},
  "emails": [{"value": "jane.smith@example.com", "primary":
true}],
  "active": true
}
```

Send the request from client PowerShell:

```
curl.exe -i -X POST "http://127.0.0.1:5004/scim/v2/Users" `
  -H "Authorization: Bearer $env:SCIM_BEARER_TOKEN" `
  -H "Content-Type: application/scim+json" `
  --data-binary "@employee.json"
```

Verified response: `HTTP/1.1 201 CREATED`. The API returned a newly generated user ID and `Location` pointing to that resource. Jane's ID in the lab was `27ed20f2-94cf-4654-9385-805bc8e8c01a`.

Why 201 matters: It tells the client the server created a new resource. It is different from 200, which normally indicates a successful retrieval or update.

7. Read users with GET

List users:

```
curl.exe -i -H "Authorization: Bearer $env:SCIM_BEARER_TOKEN"
http://127.0.0.1:5004/scim/v2/Users
```

The verified `ListResponse` reported `totalResults: 2` and included **Jane Smith** (`active: true`) and **John Doe** (`active: false`). The collection response included `Resources`, `itemsPerPage`, and `startIndex` fields.

Retrieve one user by ID:

```
curl.exe -i -H "Authorization: Bearer $env:SCIM_BEARER_TOKEN"
"http://127.0.0.1:5004/scim/v2/Users/
27ed20f2-94cf-4654-9385-805bc8e8c01a"
```

Verified: `200 OK` with Jane's user resource. Using a stable ID means a user's record can be located even if their name changes.

8. Update Jane with PATCH



Scenario: Jane's surname changes from **Smith** to **Johnson**. We update only one attribute rather than deleting and recreating the account.

`update_employee.json`:

```
{
  "schemas":
["urn:ietf:params:scim:api:messages:2.0:PatchOp"],
  "Operations": [
    {"op": "replace", "path": "name.familyName", "value":
"Johnson"}
  ]
}
```

PATCH request:

```
curl.exe -i -X PATCH "http://127.0.0.1:5004/scim/v2/Users/
27ed20f2-94cf-4654-9385-805bc8e8c01a" `
  -H "Authorization: Bearer $env:SCIM_BEARER_TOKEN" `
  -H "Content-Type: application/scim+json" `
  --data-binary "@update_employee.json"
```

Verified: `HTTP/1.1 200 OK`; the returned user contained `"familyName":"Johnson"` and remained active.

What Flask did internally

1. Validated the token and `PatchOp` schema.
2. Inspected the `replace` operation and `name.familyName` path.
3. Validated that the replacement was a nonempty string.

4. Looked up the target user by UUID.
5. Ran a parameterized SQL `UPDATE` on the `familyName` column.
6. Committed the transaction and returned the updated user resource.

Critical troubleshooting insight: Our initial PATCH returned **400 Unsupported PATCH path** because the handler did not recognize `name.familyName`. We added **both** the attribute validation branch and the database update branch. Merely accepting the JSON path without updating SQLite would not complete the task.

Illustrative SQL (the actual application used parameter placeholders):

```
UPDATE users SET familyName = 'Johnson'
WHERE id = '27ed20f2-94cf-4654-9385-805bc8e8c01a';
```

9. Verify changes in SQLite



A `200 OK` is useful evidence, but we also verified the persisted database record independently:

```
.\venv\Scripts\python.exe -c "import sqlite3;
db=sqlite3.connect('scim_users.db'); print(db.execute('SELECT
userName, givenName, familyName, active FROM
users').fetchall()); db.close()"
```

Actual verified output:

```
[('john.doe@example.com', 'John', 'Doe', 0),
 ('jane.smith@example.com', 'Jane', 'Johnson', 1)]
```

User	Before	After	Active
Jane	Jane Smith	**Jane Johnson**	`1` / true
John	John Doe	John Doe	`0` / false

Why the username stayed the same: We patched `name.familyName`, not `userName` or the email address. These are separate identity attributes.

10. Complete troubleshooting guide



Symptom	What it meant	How we diagnosed and fixed it
401 Unauthorized	Server did not accept bearer token	Check variable in **both** shells, identify active Flask process, restart with matching token
400 Unsupported PATCH path	Handler lacked `name.familyName` support	Add supported-path validation **and** SQLite `familyName` update
404 Not Found	Wrong URL, port or unregistered route	Run `python -m flask --app app routes`; confirm path and method
IndentationError: unexpected indent	Python block indentation invalid	Fix alignment around `elif`, `else`, and assignments; run `py_compile`
NameError: app not defined	Flask object referenced before initialization	Ensure `app = Flask(__name__)` exists before route decorators
	Wrong working directory	

Symptom	What it meant	How we diagnosed and fixed it
Python executable not recognized		`cd C:\Users\FOXTEC\entra-scim-lab` before `.\venv\Scripts\python.exe`
Old server still answering	Multiple Flask processes / ports	Inspect `Get-NetTCPConnection` and `Get-CimInstance`, stop the correct obsolete PID
Start-Process failed	Executable path resolved from wrong folder or invalid argument	Use an absolute path or start Flask from project folder
Set-Clipboard empty value error	PowerShell rejected a null/empty clipboard value	Set token directly in the session; don't rely on clipboard to initialize server secrets
PATCH success uncertain	HTTP response alone doesn't prove persistence	Query SQLite independently after PATCH

Exact troubleshooting commands

```
# Confirm project directory
Get-Location
Test-Path .\venv\Scripts\python.exe

# Validate syntax
.\venv\Scripts\python.exe -m py_compile app.py

# List registered routes
.\venv\Scripts\python.exe -m flask --app app routes

# Inspect listener and PID
Get-NetTCPConnection -LocalPort 5004 -State Listen
-ErrorAction SilentlyContinue |
  Select-Object LocalAddress,LocalPort,OwningProcess
```

```
# Inspect the Python process (replace PID)
Get-CimInstance Win32_Process -Filter "ProcessId = 27384" |
  Format-List ProcessId,ParentProcessId,Name,CommandLine
```

Safe restart approach: Stop the server from its own terminal with `Ctrl+C` whenever possible. If necessary, inspect the owning PID before using `Stop-Process -Id <PID> -Force`. Avoid killing all Python processes because other projects may be running.

HTTP status code reference

Code	Interpretation	Example
200	Success	GET /Users or PATCH /Users/{id}
201	Created	POST /Users
204	Success, no response body	Common successful DELETE response; not verified here
400	Invalid request	Unsupported PATCH path
401	Not authenticated	Wrong bearer token
404	Missing route or resource	Incorrect endpoint / unknown ID
409	Conflict	Duplicate unique username
500	Unhandled server failure	Inspect Flask traceback and logs

11. IAM lifecycle and results



Confirmed test results

Test	Outcome
Local Flask API running	**Passed**
Authenticated GET users	**200 OK**
Invalid-token rejection	**401 Unauthorized**
POST create Jane Smith	**201 Created**
GET Jane by ID	**200 OK**
PATCH Jane's surname	**200 OK**
Verify SQLite persisted surname	**Jane Johnson confirmed**
Full DELETE workflow	Not verified
Entra ID provisioning integration	Not yet performed

What this demonstrates for an IAM role: API authentication troubleshooting, SCIM payload handling, identity attribute mapping concepts, user lifecycle operations, process inspection, and persistence verification.

12. Security, limitations and next steps

This is a learning implementation. It runs on loopback HTTP, uses a development Flask server and a simple shared bearer secret. Do not expose it to the internet as-is.

For an enterprise integration, next steps include **HTTPS/TLS**, secret management and rotation, production server deployment, request logging/audit trails, more complete SCIM schema and PATCH semantics, pagination/filter testing, group provisioning, and automated tests. Only after those improvements should you attempt a securely reachable Microsoft Entra ID provisioning integration and verify attribute mappings and provisioning logs.

Suggested `.gitignore`

```
venv/
.venv/
__pycache__/
*.pyc
.env
scim_users.db
*.sqlite
*.sqlite3
```

Visual documentation notes

The **eight diagrams in this README are process explanations**, not screenshots. Each one is embedded **directly in its relevant section**, so GitHub displays the diagram while the reader follows the instructions. There is **no separate screenshot gallery or list of image links**. Keep the `images/` folder next to this README when uploading to GitHub; GitHub uses those files to display the embedded illustrations.

Author: Farhan Muntasir
Focus: Identity and Access Management · SCIM 2.0 · Python · REST APIs · Microsoft Entra ID integration preparation